
Real-World PBTs as Lean Challenges

Anonymous Author(s)

Affiliation

Address

email

Abstract

We present a benchmark for evaluating AI models on real-world formal software verification tasks. We first scrape $\approx 20k$ property-based tests (PBTs) from permissively licensed real-world Python repositories, then automatically translate them into Lean 4 specifications with `sorry` placeholders. Translating PBTs into formal specifications is challenging: it requires modeling Python semantics in Lean, bridging the gap between input/output testing and structural proofs, and handling the inherent difficulties of dependently-typed programming. We describe a three-agent LLM pipeline for transpiling PBT suites into Lean specifications, evaluate coverage and quality metrics, and provide baselines for proof completion using several automated and model-based approaches. Our benchmark aims to drive progress on the underexplored problem of AI-assisted formal verification of real-world software, which we claim is of increasing import as AI begins to produce an unprecedented percentage of the world’s code.

1 Introduction

Several AI safety proposals are premised on AI-driven formal verification (FV); this is a core hypothesis of the ARIA Safeguarding AI program and the related Guaranteed Safe AI agenda Dalrymple et al. [2024]. If FV is to play this role, it must scale to the complexities of real-world code, but today we have little evidence that such capabilities are possible. Relevant benchmarks are mostly small datasets crafted by hand Loughridge et al. [2024], larger datasets extracted from expert-driven verification projects Chakraborty et al. [2025], or are focused on advanced mathematics rather than engineering Glazer et al. [2024]. None of these are likely to be representative of the formal verification tasks required to verify real-world AI-generated software at scale.

We generate a new benchmark, FVSpec, from public uses of property-based testing (PBT). While FV itself is rarely used in software development, PBT is a ‘close sibling’ technology which has seen significant adoption. In both FV and PBT, engineers write *specifications*: logical properties that the software must obey. For example, we might require that a Python function `isort(1st)` always returns a sorted permutation of its input `1st`. The difference lies in how this property is checked. In FV, the property is proved using mathematical techniques—this results in almost-perfect confidence, but requires proof engineering by highly expert teams Dodds [2022]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering.

PBT is typically much cheaper than FV, and as a result has seen wider adoption. In fact, PBTs are typically seen as the first step toward FV. For example, in the interactive theorem prover ACL2s, any unproven theorem is automatically treated as a PBT (and tested against randomly generated inputs) until the author manages to supply a proof. Thus the idea of translating PBTs to theorems for FV is natural and, to an extent, industry-standard. We build on this intuition. First, we scrape $\approx 20k$ permissively-licensed PBTs written in Python’s Hypothesis framework MacIver et al. [2019]. Each PBT in our dataset can be seen as a small theorem statement (or *ansatz*) about a piece of Python code, which the author would like to prove or, failing that, at least test. To make it possible for an AI to

39 prove or refute these theorems, we then translate both code and PBTs to the theorem prover Lean,
 40 which is becoming the de-facto standard in AI formal verification. This approach was pioneered
 41 by Dougherty and Mehta on the FVAPPS verification benchmark Dougherty and Mehta [2025]. In
 42 FVAPPS, source programs were taken from the APPS benchmark by Hendrycks et al Hendrycks et al.
 43 [2021], translated to PBTs, and then lifted to theorems. We apply the same process at scale to PBTs
 44 found in the wild.

45 We use AI-driven translation to generate Lean programs and theorems at scale. LLMs are quite good
 46 at program translation, and FVAPPS has shown that this approach is practical for constructing Lean
 47 theorems. Although in some cases the translations may be imperfect, lessening the importance of
 48 any proof or refutation of the resultant theorem from the perspective of the author of the originating
 49 PBT, it is nevertheless the case that we produce high-quality and realistic FV challenges for AI
 50 benchmarking. The result is tens of thousands of verification challenges, each consisting of (1) a
 51 program and (2) a desired specification, both written in Lean. Unlike existing FV benchmarks, each
 52 problem corresponds to properties of real-world software written by engineers with no guaranteed
 53 formal verification experience.

54 **TODO:** Summary of results.

55 2 Task Definition

56 Each problem in the FVSpec benchmark consists of three components: (1) the original Python
 57 property-based test, (2) a Lean theorem corresponding to the PBT, with a sorry placeholder where the
 58 proof should go, and (3) a Lean implementation of the functions referenced in the theorem. The AI’s
 59 task is to complete the proof by replacing sorry with a valid Lean proof term. Each problem also
 60 includes a URL pointing to the original source code and license information.

61 2.1 Running Example

62 Consider a Hypothesis test asserting the Pythagorean trigonometric identity $\sin^2(x) + \cos^2(x) = 1$,
 63 for a trigonometric library trg:

```
64 from hypothesis import given, strategies
65
66
67 @given(x=strategies.floats(
68     min_value=-sys.float_info.max / 4,
69     max_value=sys.float_info.max / 4,
70     allow_infinity=False))
71 def test_property(x):
72     assert trg.sine(x) ** 2 + trg.cosine(x) ** 2 == pytest.approx(1)
```

74 Our pipeline translates this into a Lean implementation:

```
75 namespace Fvspec.Impl
76
77
78 def to_radians (degrees : Float) : Float :=
79     degrees * Float.pi / 180.0
80
81 def sine (theta : Float) : Float :=
82     Float.sin (to_radians theta)
83
84 end Fvspec.Impl
```

86 And a corresponding Lean specification:

```
87 import Fvspec.Impl
88 open Fvspec.Impl
89
90
91 def cosine (theta : Float) : Float := sorry
92
93 theorem sine_cosine_pythagorean (x : Float) :
94     (sine x) * (sine x) + (cosine x) * (cosine x) = 1 := sorry
```

96 The AI must fill in both sorry placeholders with valid Lean proof terms.

97 **TODO**– Explain how we do not faithfully translate the original `trg` library into Lean, with all
98 dependencies and so forth, rather, we utilize Lean’s built-in `sine` and `cosine` functions. This is
99 representative of our general attitude which is that our primary goal is to produce equally interesting,
100 realistic problems to benchmark AI on, while actually proving properties about this real-world
101 software to help these specific code authors is just a secondary goal.

102 2.2 PBT vs. Proof

103 In PBT, a property is checked by randomly generating inputs and testing them—the process is push-
104 button but provides only statistical confidence. In FV, the property is proved using computer-aided
105 symbolic logic, providing near-perfect confidence but requiring proof engineering.

106 Translating a PBT into a formal specification thus involves a shift from input/output testing to proofs
107 over the structure of the program. Our pipeline filters nonsensical theorems, but any discrepancies
108 that remain will not affect the usefulness of the dataset. Because the Lean theorem prover provides a
109 ‘perfect’ oracle, every theorem we generate is a useful challenge problem, even if it differs from the
110 original PBT. In fact, some PBTs in our input dataset will be naturally false due to undetected edge
111 cases.

112 **TODO**– Explain what we mean by nonsensical theorems.

113 **TODO**– Explain to what degree Lean can be viewed as a perfect oracle.

114 2.3 Challenges

115 Formal verification is unusually difficult to benchmark for two reasons. First, just as with advanced
116 mathematics, formal verification tasks are highly varied, and constructing interesting problems
117 requires considerable expertise. FV tasks also vary in difficulty—they can be as shallow as checking
118 the type-safety of simple functions, or as deep as proving an unsolved mathematical theorem such as
119 the Collatz Conjecture. Our benchmark avoids the need to select and hand-build challenge problems
120 by translating naturally occurring PBT problems written by real-world engineers.

121 Second, formal verification is little-used outside academia. As a result, there are few natural datasets,
122 and those that exist are fragmented between multiple FV tools (e.g. SAW/Cryptol for cryptographic
123 systems, ACL2/Coq for traditional software verification, Lean for mathematics, SPIN/TLA+ for
124 protocol analysis)—or are privately held, closely guarded corporate IP (such as the FV work done by
125 companies like Intel or Synopsys). Public FV examples are often pedagogical exercises for which
126 the proof exists in many different forms in the training set. Our benchmark solves this problem
127 by translating from a ‘close sibling’ domain, PBT, where many more datapoints exist outside of
128 pedagogy or academic research. As a result, most theorems in our benchmark will have never been
129 verified by anyone.

130 **TODO**– A reviewer will ask us to substantiate this claim.

131 Beyond these structural challenges, AI-driven translation at scale introduces practical difficulties.
132 Each failed pipeline run is expensive—a single sample may require multiple LLM calls with iterative
133 repair, and a bug in the pipeline infrastructure can invalidate an entire batch. Quality improvements
134 must therefore be targeted: we identify specific failure modes in the output (e.g. type restrictions,
135 missing dependencies, trivial stub theorems) and address them incrementally rather than re-running
136 the full dataset. We initially considered extending the benchmark to TypeScript PBTs, but found that
137 the PBT ecosystem in TypeScript is far less mature, and the additional coverage would not justify the
138 engineering effort.

139 3 Pipeline

140 Our generation pipeline is modelled on FVAPPS Dougherty and Mehta [2025], but starts from
141 real-world Hypothesis property-based tests (PBTs) rather than synthetic programming puzzles. For
142 each sample, we (i) recover the Python function under test (FUT) and dependencies, (ii) translate
143 the code to a computable Lean implementation, (iii) translate the PBTs into Lean theorems, and (iv)
144 score and filter the resulting artifacts (Figure 1).

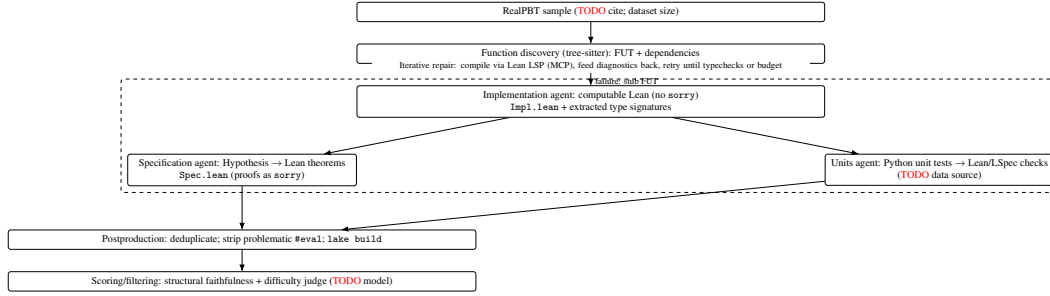


Figure 1: FVSpec pipeline: discover FUT+deps, translate to computable Lean, generate specs, repair via Lean LSP, then postprocess and score.

145 3.1 Function discovery

146 For each PBT, we use tree-sitter to extract the Python source code of the FUT and any locally-defined
 147 dependencies. This function discovery step succeeds for **TODO** % of samples; when it fails, we
 148 generate a stub FUT with generic type parameters to avoid producing trivial theorems.

149 3.2 LLM-based transpilation

150 We translate each sample using three agents.

151 **Implementation agent** The implementation agent runs first, translating the FUT and its dependen-
 152 cies into fully computable Lean definitions. No `sorry` placeholders are permitted, so the output must
 153 typecheck and evaluate. Dependencies are formalized separately and then merged with deduplica-
 154 tion into a single `Impl.lean` file. On success, we additionally extract Lean type signatures for all
 155 formalized functions and provide them as context to downstream agents. If the agent exhausts its
 156 retry budget without producing typechecking code, downstream stages still run, but without a verified
 157 implementation to build on.

158 **Specification and units agents** Next, the specification agent and units agent run in parallel.
 159 The specification agent translates Hypothesis tests into Lean theorems that import the formalized
 160 implementation; proofs are left as `sorry`. The units agent translates any available Python unit tests
 161 into Lean/LSpec checks. The units output is included as auxiliary validation, but does not gate
 162 inclusion of the specification (**TODO** confirm policy).

163 3.3 Iterative repair against Lean

164 All agents operate in an iterative repair loop. At each step, we check the generated Lean code against
 165 the compiler via the Lean LSP (accessed through an MCP tool server). If the code does not typecheck,
 166 we feed the compiler diagnostics back to the agent and prompt for a revised translation. This loop
 167 continues until the code typechecks or a retry budget is exhausted. We validate final compilation with
 168 a full `lake build`.

169 3.4 Cleanup and quality evaluation

170 We apply postproduction passes to remove known failure modes (e.g., stray `#eval` fragments that can
 171 pass in isolation but break a project build) and to merge/deduplicate translated dependencies across
 172 files.

173 The resulting specification/implementation pairs are then scored and filtered. We compute *structural*
 174 *faithfulness* programmatically by comparing the generated Lean artifacts to the source PBTs. The
 175 metric decomposes into sub-scores for parameter coverage, type correspondence, strategy coverage,
 176 assertion coverage, and dependency coverage, producing an overall score (**TODO** define weights;
 177 add exact formula). We also assign a *difficulty* rating using an LLM judge (**TODO** specify model and
 178 prompt), intended to approximate the proof effort required on a 1–10 scale.

3.5 Input PBT corpus

Source PBTs are drawn from the `Benchify/realpbt` dataset ?; Section ?? describes how the corpus was scraped, fork-deduplicated, and cleaned in detail. The working set after deduplication is 11,039 PBTs across 333 repositories.

4 Dataset Characterization

4.1 Input corpus: scraping and deduplication

Scraping. Source PBTs come from the `Benchify/realpbt` dataset ?, assembled by a custom Python scraper that operates on the GitHub dependency graph of the Hypothesis library. The scraper takes a pre-collected list of $\approx 54k$ dependent repositories, iterates over them asynchronously with rate-limit-aware GitHub REST API backoff, shallow-clones each repository, and uses `ripgrep` to locate files containing the `@given` decorator. For each such file, it parses the Python AST to extract all PBT functions—identified as function definitions whose decorator list includes a call to `given`—and performs a transitive static call-graph traversal to recover the minimal closure of locally-defined helpers called by each PBT. Repository metadata (name, URL, license, stars, forks, commit hash) and extracted PBTs (source code, dependency names and their source) are persisted to PostgreSQL; repos whose commit hash was already processed are skipped, enabling incremental re-scraping. Only repositories whose license matches a permissive allowlist (MIT, Apache-2.0, BSD variants, ISC, Unlicense, CC0, BlueOak-1.0.0) contribute PBTs to the released dataset.

Fork deduplication. The raw scrape yielded 54,345 PBTs, but GitHub’s dependency graph surfaces many forks of the same upstream (e.g., hundreds of forks of `pytorch` each carrying an identical Hypothesis test suite). We deduplicate by grouping repositories whose names share the same base component, selecting a canonical representative per group (highest star count, breaking ties by PBT count), and retaining PBTs from canonical repos plus any PBTs unique to forks (identified by SHA-256 of whitespace-normalized code). After fork deduplication the corpus contains 11,039 PBTs across 333 repositories.

4.2 Post-production pipeline

After the formalization agent produces Lean artifacts, a multi-stage post-production pipeline processes the raw run outputs into a final curated dataset. The stages are: `merge` $\rightarrow$ `validate` $\rightarrow$ `unify` $\rightarrow$ `dedup` $\rightarrow$ `enrich` $\rightarrow$ `grade` $\rightarrow$ `metrics`. Each stage is an independent CLI command (`uv run postprod <stage>`) and the full pipeline state is checkpointed so it can be resumed after interruption.

Merge. Multiple independent benchmark runs (identified by timestamped run directories) are merged into a single JSONL file. Each sample directory contains `datapoint.json` (source PBT metadata), `qa.json` (agent outputs and metrics), and `Spec.lean/Impl.lean` (generated Lean artifacts). When the same `sample_id` appears in multiple runs, the merge step keeps the highest-quality copy, scored by: (1) success flag, (2) overall structural faithfulness, (3) theorem count, (4) presence of unit tests, (5) implementation autoformalization success, (6) specification signature success. Git commit hash, model, run timestamp, and Lean toolchain version are recorded as provenance fields.

Validate. Because an agent can produce code that typechecks in isolation but fails under a full project build, we validate compiled samples by instantiating each `Spec.lean/Impl.lean` pair into a `lake build` project and running the Lean compiler. Validation runs up to 400 samples by default (yielding $\pm 5\%$ accuracy at 95% CI) or all samples with `-all`. Only samples that pass `lake build` are retained in the dataset; failures are logged but dropped.

Unify. Our dataset was produced in two pipeline generations (*feb03* and *apr08*) with schema drift between them. The unify step resolves these differences: it renames `radon` $\rightarrow$ `realpbt_radon`, backfills the structured dependencies list from *feb03*’s legacy parallel JSON-stringified arrays (filling null for fields not captured in that schema), drops 100%-null dead fields (`lean_metrics.tests.structure`, `lean_metrics.tests.complexity`,

227 `metrics_metadata.tests_available`), and reassigns fresh monotone `sample_ids` 1..N with
228 `feb03` rows first. Each row carries a run tag indicating its source generation.

229 **Code-level deduplication.** The same PBT may have been formalized in multiple runs (e.g., because
230 sampling overlap exists between run index ranges, or because a PBT appeared in both `feb03` and
231 `apr08`). This step groups samples by SHA-256 of `realpbt_code` and keeps the single best repre-
232 sentative per group (using the same quality scoring as the merge step), separately tracking cross-run
233 vs. within-run duplicates for audit.

234 **Enrich.** Every sample is annotated with: `code_hash` (SHA-256 of `realpbt_code`),
235 `is_canonical` (whether it is the best formalization of its PBT), `formalization_rank` (1-based
236 rank within the group, 1 = canonical), `num_formalizations` (how many runs produced a formal-
237 ization of this PBT), and `pareto_dominated` (whether a strictly better sample exists on all five
238 structural faithfulness sub-metrics: parameter coverage, type correspondence, strategy coverage,
239 assertion coverage, and dependency coverage). Stale v1 grader fields with inconsistent coverage are
240 also stripped here, and columns are reordered for clean HuggingFace presentation.

241 **Difficulty grading.** Each sample is scored for formalization difficulty by Claude Haiku, producing
242 a binary easy/hard label (`difficulty_binary`) with a calibrated confidence score and a brief
243 reasoning trace. The grader uses a structured prompt including the PBT source code, the generated
244 Lean spec, and contextual information about the formalization strategy. Grading is resumable: a
245 checkpoint file accumulates completed grades so a killed run restarts where it left off.

246 **Lean metrics.** A final metrics pass runs `radon` on the Python source and a custom Lean AST
247 analyzer on `Spec.lean/Impl.lean` to populate `lean_metrics` (total lines, sorry count, axiom
248 count, spec structure and complexity fields including Halstead volume and difficulty, theorem/def
249 counts).

250 4.3 Corpus statistics

251 Table ?? summarizes the post-deduplication input corpus. The star and fork distributions are heavily
252 right-skewed: the median repository has 0 stars, reflecting that most real-world Hypothesis users work
253 on small or private-to-GitHub projects. The median PBT is 13 lines with a cyclomatic complexity of
254 2, consistent with the concise, assertion-focused style typical of Hypothesis tests.

255 Figures illustrating the distributions of PBTs per repo, stars, forks, and complexity are gener-
256 ated by `comms/graphs/src/graphs/realpbt.py` and are reproducible via `make realpbt` in
257 `comms/graphs/`.

258 Table ?? compares the RealPBT input corpus with the contemporaneous Hypothesis Corpus of DeVoe
259 et al. ? (see also Section ??).

260 5 Results

261 We present a characterization of the FVSpec benchmark along six axes: dataset scale, source code
262 properties, translation quality, output complexity, pipeline cost, and difficulty stratification. All
263 quantitative claims in this section correspond to programmatically generated figures; we report the
264 dataset-level statistics from the public HuggingFace release and direct the reader to the accompanying
265 visualizations for distributional detail.

266 5.1 Dataset Scale

267 The released dataset contains **9,415 Lean 4 formalization challenges** derived from **2,772 unique**
268 **Python property-based tests**. Two pipeline runs contribute to the dataset: an initial run (`feb03`,
269 5,979 samples, Claude Sonnet) and a second run (`apr08`, 3,436 samples, Claude Sonnet 4.6). Because
270 the same PBT may be formalized in multiple ways across runs, each unique PBT has between 1
271 and 15 formalizations. Sixty-five percent of multi-formalization groups exhibit a Pareto front larger
272 than one: no single formalization dominates on all structural faithfulness sub-metrics simultaneously,

Table 1: Summary statistics for the post-deduplication RealPBT input corpus.

Metric	Value
PBTs (pre-dedup, HuggingFace)	54,345
PBTs (post-dedup, working set)	11,039
Unique repositories	333
Median PBTs per repo	6
Mean PBTs per repo	33.2
Max PBTs in a single repo	956
Repos with only 1 PBT	61
Median repo stars	0
Mean repo stars	39
Repos with ≥ 100 stars	10
Repos with $\geq 1,000$ stars	1
Median PBT LOC	13
Mean PBT LOC	17.2
Median cyclomatic complexity	2.0
Mean cyclomatic complexity	3.3
MIT-licensed repos	148
Apache-2.0-licensed repos	66
BSD-3-Clause-licensed repos	25

Table 2: Comparison of FVSpec’s RealPBT corpus with the Hypothesis Corpus ?.

	Ours (RealPBT)	Hypothesis Corpus
PBTs (pre-dedup)	54,345	28,928
PBTs (post-dedup)	11,039	n/a
Repositories (post-dedup)	333	1,529
Dependency source code	✓	×
Radon code metrics	✓	×
Natural-language summary	✓	×
Permissive-license filter	✓	×
@settings configuration	×	✓
Runtime & entropy information	×	✓
Line coverage data	×	✓
Repository overlap	not computed (corpus gated)	

confirming that the multiple-formalization design captures genuinely complementary aspects of each PBT.

5.2 Python Source Characteristics

Figure 2 characterizes the input PBTs along three axes: lines of PBT code, average cyclomatic complexity (Radon), and maintainability index. The source tests span a wide range of complexity, from trivial single-assertion tests to multi-strategy tests exercising complex library APIs. This diversity is inherited directly from real-world engineering practice and is not curated or filtered for difficulty.

5.3 Translation Quality

We evaluate translation quality using *structural faithfulness*, a composite metric that decomposes into five sub-scores: parameter coverage, type correspondence, strategy coverage, assertion coverage, and dependency coverage (described in the preceding section). Figure 3 shows the distribution of the overall score and the mean of each sub-score across the dataset. The overall faithfulness distribution

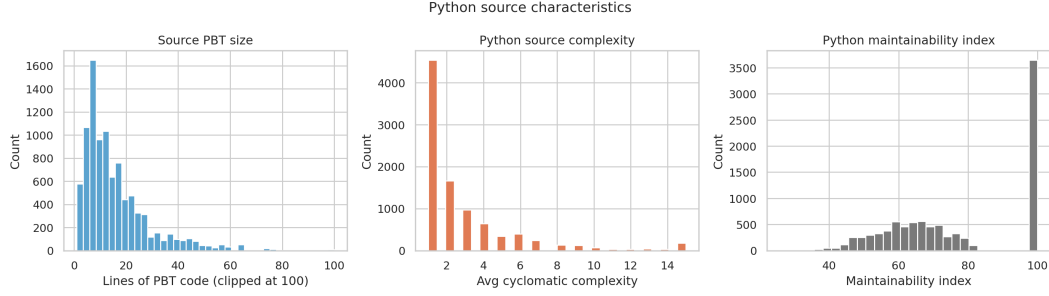


Figure 2: Python source characteristics of the input PBTs: lines of code, cyclomatic complexity (Radon), and maintainability index.

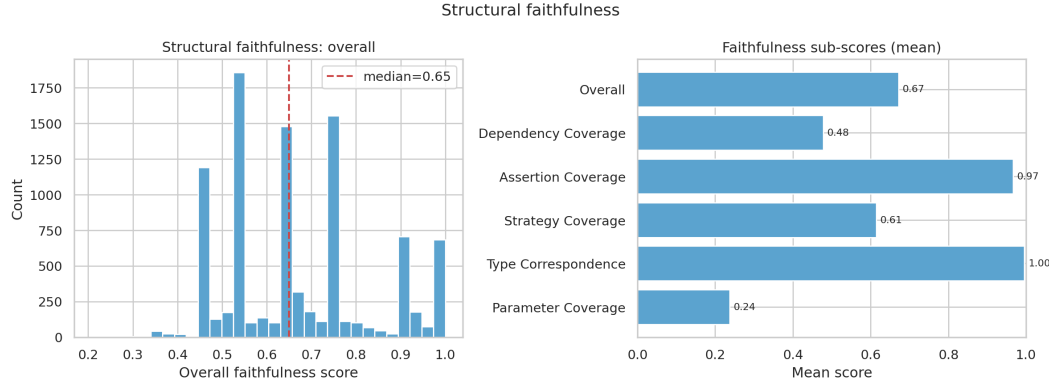


Figure 3: Structural faithfulness: distribution of the overall composite score (left) and mean sub-scores (right).

286 exhibits a clear mode above 0.5, indicating that the majority of translations preserve the essential
 287 structure of the source PBT, though a long tail of lower-quality translations remains.

288 Figure 4 breaks down the dataset by implementation level and shows mean faithfulness conditioned
 289 on implementation level. Samples where the implementation agent successfully produced fully
 290 computable Lean code achieve higher faithfulness on average than those where it fell back to stubs,
 291 confirming that upstream translation quality propagates to specification fidelity.

292 5.4 Lean Output Complexity

293 Figure 5 characterizes the generated Lean artifacts: total lines of Lean code, number of theorems per
 294 sample, and number of sorry placeholders. The median sample contains multiple theorems, each
 295 requiring an independent proof. The sorry count tracks the theorem count closely, as expected from
 296 our pipeline design (each theorem is emitted with a sorry placeholder).

297 5.5 Pipeline Cost

298 Figure 6 reports the per-sample pipeline cost in three dimensions: wall-clock time, token usage, and
 299 number of agent turns. The distributions are right-skewed, reflecting the iterative repair loop: most
 300 samples compile within a few turns, but a tail of difficult translations requires many repair iterations
 301 before the Lean LSP reports success or the budget is exhausted.

302 5.6 Difficulty Stratification

303 Each sample is graded for proof difficulty by Claude Haiku 4.5, which assigns a binary label (*easy* or
 304 *hard*) along with a confidence score. Figure 7 shows the resulting split and the distribution of grader

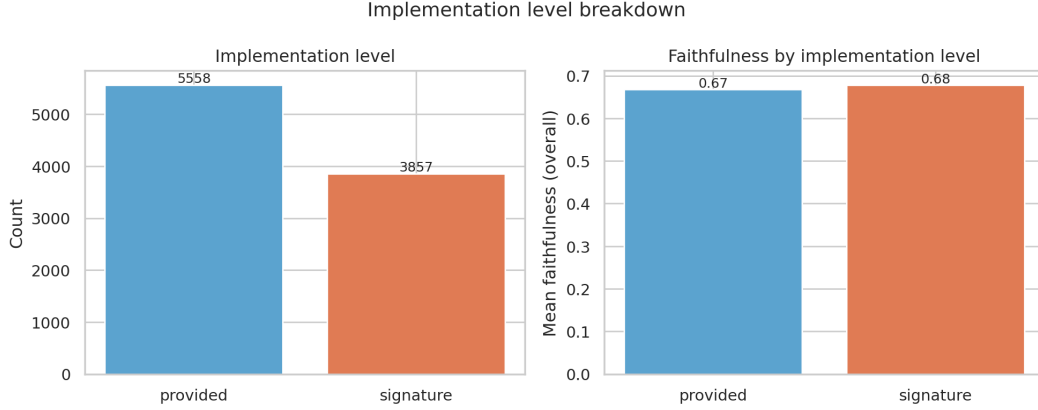


Figure 4: Implementation level breakdown: count by level (left) and mean overall faithfulness by level (right).

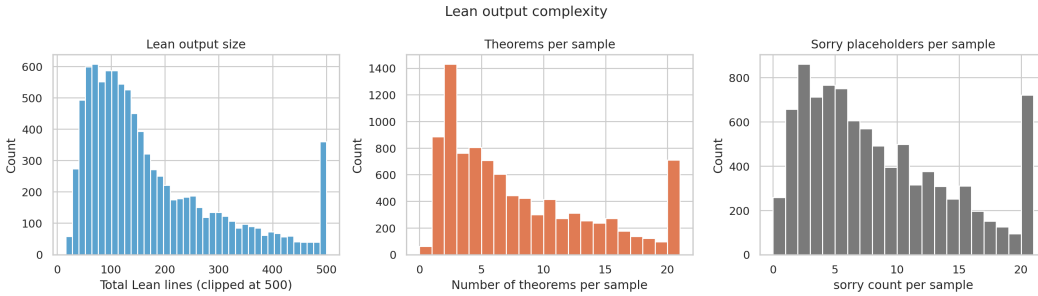


Figure 5: Lean output complexity: total lines (left), theorems per sample (center), and sorry placeholders per sample (right).

confidence. The benchmark contains a substantial proportion of both easy and hard problems, and the grader expresses high confidence for the majority of assessments.

Figure 8 examines the relationship between difficulty and two output characteristics: structural faithfulness and Lean output size. Hard problems tend to produce longer Lean outputs, consistent with the intuition that more complex source PBTs lead to more elaborate formalizations and more challenging proofs.

5.7 Baselines

TODO Baseline evaluations are in progress. We evaluate four frontier models—Claude Sonnet 4.6, Claude Opus 4.6, GPT-5.4-mini, and Gemini 2.5 Pro—on a stratified sample of 75 problems (50% easy, 50% hard). Each model is given access to the Lean LSP via MCP tools and scored on two metrics: a binary *proved* score (zero sorry remaining and lake build succeeds with no sorry warnings) and *partial credit* (fraction of sorry placeholders removed, clamped to $[0, 1]$). Results will be reported as per-bucket (easy/hard) and aggregate prove rates, with a comparison table across models.

6 Threats to Validity

We note that the theorems we construct may be false—this is desirable because the AI has the opportunity to contradict the theorem (and perhaps identify a counter-example). Because the Lean theorem prover is a so-called foundational tool, once a theorem is verified or contradicted, we have near-perfect confidence the answer is correct. However, because our specifications are translated rather than hand-written, some may not faithfully represent the original PBT. Our structural faithfulness

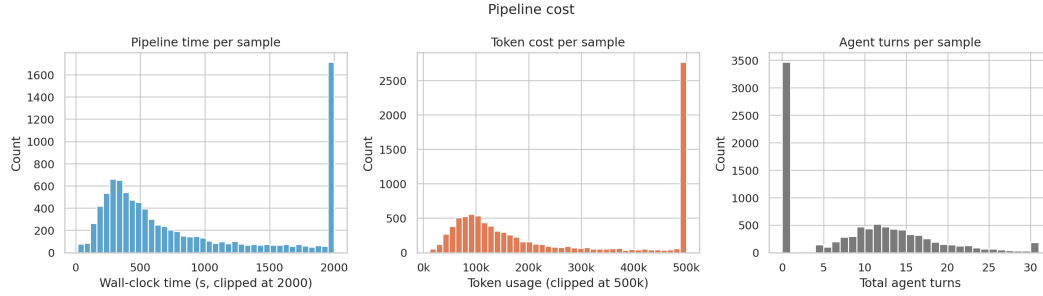


Figure 6: Pipeline cost per sample: wall-clock time (left), token usage (center), and agent turns (right).

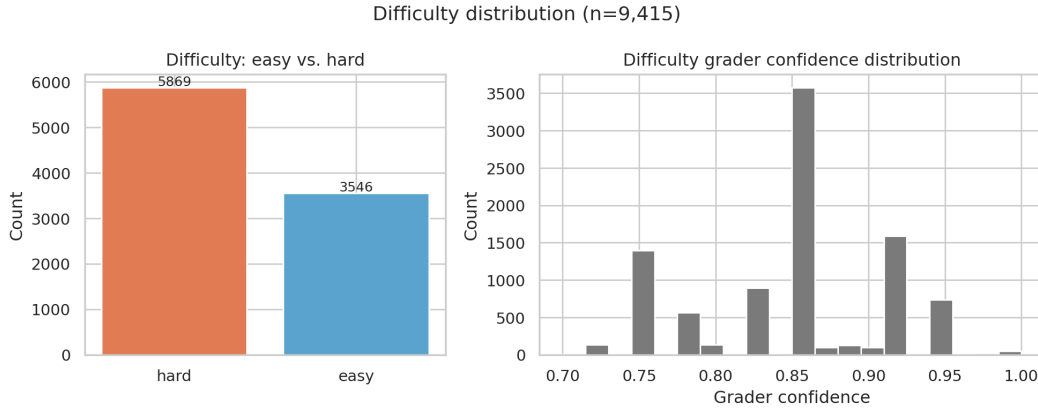


Figure 7: Difficulty distribution: easy vs. hard counts (left) and grader confidence distribution (right). n shown in title.

metric provides an objective measure of translation quality, but it is a heuristic: a high score does not guarantee semantic equivalence, and a low score does not necessarily indicate a useless problem.

Our benchmark does not include human baselines. We expect the dataset to exhibit a wide range of difficulty, including many problems that are trivial for current AIs, and many harder problems, including some insurmountable even to world-expert human teams. Without human expert performance data, we cannot directly calibrate this difficulty distribution.

The pipeline’s reliance on LLM-driven translation means that benchmark quality is bounded by the capabilities of the translation model. Samples where function discovery failed (8%) produce stub-based theorems that may be less representative. We mitigate this through postproduction filtering and the structural faithfulness metric, but some lower-quality samples may remain.

7 Related Work

Our work extends FVAPPS by Dougherty and Mehta [2025], which pioneered AI-driven translation of coding problems into Lean theorems. In FVAPPS, source programs were taken from the APPS benchmark Hendrycks et al. [2021], translated to PBTs, then lifted to theorems. We apply the same process at scale to PBTs found in the wild, rather than synthetic coding challenges.

Lean verification benchmarks. A growing number of benchmarks target Lean specifically. mini-CodeProps Brandfonbrener et al. [2024] provides 177 program specifications translated from TIP (Tons of Inductive Programs), finding that current LLM provers fail on medium and hard properties. CLEVER Materzok et al. [2025] offers 161 curated problems sourced from HumanEval, testing end-to-end verified code generation. Verina Wu et al. [2025b] covers all three foundational tasks—code generation, specification generation, and proof generation—across 189 manually curated

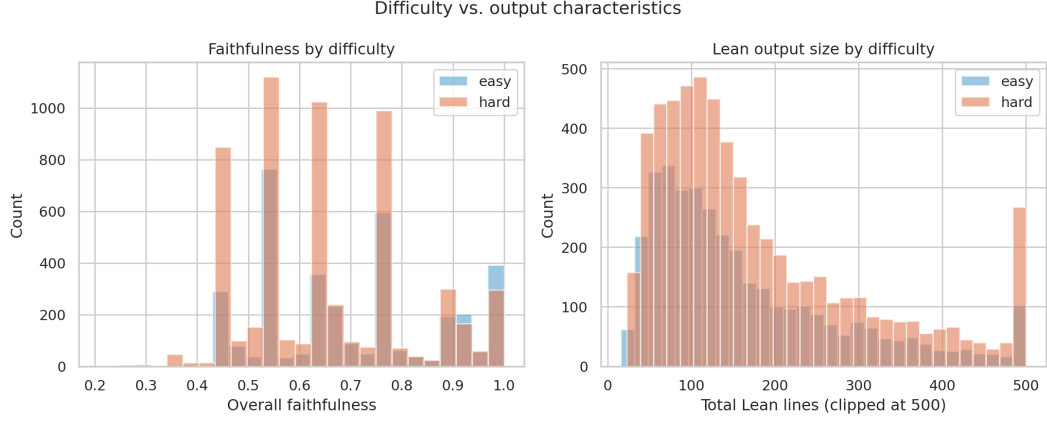


Figure 8: Difficulty vs. output characteristics: overall faithfulness (left) and total Lean lines (right), stratified by difficulty.

coding tasks. VeriBench Stechly et al. [2025] translates Python code with documentation into Lean implementations, specifications, and unit tests (113 tasks). The Vericoding Benchmark Bursuc et al. [2025] supports Lean, Rust/Verus, and Dafny, reporting success rates from 27% (Lean) to 82% (Dafny). VeriEquivBench Ma et al. [2025] uses LeetCode transformations to evaluate verifiable agents on novel data without contamination. These benchmarks are valuable but share a common limitation: their specifications derive from synthetic coding problems or pedagogical exercises, not from properties written by practicing engineers.

Other verification tool benchmarks. DafnyBench Loughridge et al. [2024] provides Dafny verification tasks. VerusBench Chen et al. [2024] collects 150 proof tasks spanning Dafny, SV-COMP, and Verus. SV-COMP Beyer [2025] is the standard competition for software verifiers, with over 33,000 C tasks, but targets automated verification rather than interactive theorem proving. InvBench Wu et al. [2025a] focuses specifically on loop invariant synthesis. Chakraborty et al. Chakraborty et al. [2025] explore neural synthesis for SMT-assisted proof-oriented programming in F*.

Interactive theorem proving benchmarks. LeanDojo Benchmark 4 Yang et al. [2024] provides 122,517 theorems from Mathlib4. CoqStoq Thompson et al. [2025] collects 196,929 theorems from 2,226 GitHub Coq repositories. These are primarily mathematical rather than software-oriented, and their theorems exist in the training data of most frontier models.

Mathematics benchmarks. FrontierMath Glazer et al. [2024] evaluates advanced mathematical reasoning. While formal verification and mathematics share proof infrastructure, the skills required differ substantially: software verification demands modelling of program semantics, library APIs, and computational behavior, rather than abstract mathematical reasoning.

FVSpec differs from existing benchmarks in three ways. First, our specifications originate from real-world property-based tests written by practicing engineers, not synthetic problems or pedagogical exercises. Second, the source PBTs were never formally verified, so most theorems in our benchmark will not appear in any model’s training data. Third, we operate at a larger scale than existing Lean verification benchmarks, with thousands rather than hundreds of problems. Our work is motivated by the broader context of AI safety proposals premised on AI-driven formal verification, including the Guaranteed Safe AI agenda Dalrymple et al. [2024].

The Hypothesis Corpus. DeVoe et al. recently released the Hypothesis Corpus ?, a dataset of 28,928 Hypothesis tests from 1,529 repositories, collected by the Hypothesis development team. It is the only other large-scale collection of real-world Hypothesis PBTs that we are aware of, and was assembled with complementary goals: whereas our dataset is oriented toward formal verification (each PBT is paired with dependency source code, Radon metrics, and a natural-language summary, and is filtered to permissively-licensed repos for downstream use), theirs is oriented toward PBT engine research and includes rich runtime information—entropy consumed during generation, test

outcome, assume/filter/event/note/target call values, and line coverage—that our scraping pipeline does not collect. A programmatic comparison is limited by the Hypothesis Corpus being gated on HuggingFace (requiring authentication), so we cannot directly compute repository overlap. At the level of schema, our dataset uniquely provides resolved dependency source code and per-PBT Radon code quality metrics, while theirs uniquely provides @settings configuration and execution-time data; neither is a strict superset of the other. Table ?? in Section ?? gives a full side-by-side comparison.

8 Conclusion / Future Work

Several extensions are possible:

TODO: Add examples from other languages with significant usage of PBT, e.g. Haskell/QuickCheck, C++/RapidCheck. Translate our benchmark to other theorem proving systems, e.g. Coq, Isabelle, F*, etc. More advanced website features, such as prediction markets about when the benchmark will be saturated, and a pipeline for submitting code to the leaderboard which we then persist into a training or SFT dataset. Our benchmark will also be useful for developing new techniques in program repair and synthesis. If a property is disproven, this creates a new interesting challenge, namely to synthesize a patch to the original function such that the theorem now holds. We hypothesize that many such bugs will be particularly subtle and interesting edge cases since, by definition, in most cases they were not detected by the original developer’s PBT harness.

References

- D. Beyer. State of the art in software verification and witness validation: SV-COMP 2025. In *Proceedings of the 31st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2025.
- D. Brandfonbrener, L. Beurerkellner, D. Romero, N. Amin, and M. Vechev. miniCodeProps: a minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.
- S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley, A. Bizid, Q. Dougherty, M. Zhao, M. Tan, and M. Tegmark. A benchmark for vericoding: formally verified program synthesis, 2025. URL <https://arxiv.org/abs/2509.22908>.
- S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- H. Chen et al. VerusBench: A benchmark for automated verification of Rust with Verus. *arXiv preprint arXiv:2409.13082*, 2024.
- D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy, B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.
- M. Dodds. Formally verifying industry cryptography. *IEEE Security & Privacy*, 20(3):65–70, 2022. doi: 10.1109/MSEC.2022.3153035.
- Q. Dougherty and R. Mehta. Proving the coding interview: A benchmark for formally verified code generation. In *Proceedings of the LLM4Code Workshop at the 47th International Conference on Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.
- E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S. Denain, A. Ho, E. de Oliveira Santos, O. Järvinen, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla, Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla, and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.

427 D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He,
428 D. Song, and J. Steinhardt. Measuring coding challenge competence with APPS. In *Proceedings*
429 *of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

430 C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu,
431 N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv*
432 *preprint arXiv:2406.08467*, 2024.

433 X. Ma et al. VeriEquivBench: Evaluating verifiable agents on novel data without contamination.
434 *arXiv preprint arXiv:2510.06296*, 2025.

435 D. R. MacIver, Z. Hatfield-Dodds, and many other contributors. Hypothesis: A new approach to
436 property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.
437 01891.

438 M. Materzok, W. Szulc, M. Tabiś, J. Kozak, and B. Piotrowski. CLEVER: A benchmark for
439 closed-loop end-to-end formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.

440 K. Stechly et al. VeriBench: Benchmarking end-to-end formal verification. In *AI4Math Workshop at*
441 *ICML*, 2025. URL <https://openreview.net/forum?id=rWkGFmnSN1>.

442 K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira, S. Lerner,
443 and E. First. Rango: Adaptive retrieval-augmented proving for automated software verification,
444 2025. URL <https://arxiv.org/abs/2412.14063>.

445 T.-Y. Wu et al. InvBench: A benchmark for loop invariant synthesis. *arXiv preprint arXiv:2509.21629*,
446 2025a.

447 W.-D. Wu, K. Wang, R. Mangal, and I. Dillig. Verina: Benchmarking formally verified code
448 generation. *arXiv preprint arXiv:2505.23135*, 2025b.

449 K. Yang, A. Swope, A. Gu, R. Chalapathy, P. Song, S. Lember, A. Thomson, and A. Anandkumar.
450 LeanDojo: Theorem proving with retrieval-augmented language models. *Advances in Neural*
451 *Information Processing Systems (NeurIPS)*, 2024.